

Чек-лист по API проверкам

Надежда Дудник
2023 - 2026

Об авторе



Меня зовут Надежда Дудник, я являюсь автором данного чек-листа.

- Ведущий инженер по тестированию ПО с многолетним опытом.
- Ментор в сфере тестирования ПО.

Моя страсть — это передача практических знаний и навыков, создание эффективных тестовых задач для закрепления материала и обмен ценным опытом.

- Активно развиваю профессиональный блог по тестированию ПО, где делюсь экспертизой.
- Провожу обучение и консультирую специалистов отрасли.

Для чего этот чек-лист

Представленный чек-лист разработан, чтобы помочь вам:

- **Расширить охват тестирования:** Создавать более полные тестовые сценарии для API.
- **Освоить виды проверок:** Четко различать позитивные и негативные проверки API.
- **Систематизировать знания:** Упорядочить и углубить навыки в области API-тестирования.
- **Применять знания на практике:** Эффективно решать тестовые и проектные задачи с использованием API-тестирования.
- **Изучить особенности API:** Ознакомиться с ключевыми аспектами тестирования API.

Контакты и ресурсы

- [Телеграм-канал Protestinginfo](#): Делюсь экспертизой и актуальной информацией.
- [Сайт Protestinginfo](#): Моя основная платформа со статьями и ресурсами.
- [Блог в Instagram](#): Дополнительные материалы и общение.
- [Канал для оповещений](#): Получайте информацию о старте курсов и практических занятий.

Чек-лист для позитивных проверок API

Позитивные проверки API направлены на подтверждение того, что API функционирует корректно при получении ожидаемых и валидных входных данных. Они жизненно важны для обеспечения базовой работоспособности и соответствия системы требованиям.

Используйте этот чек-лист для создания исчерпывающих тестовых сценариев, систематизации знаний и эффективного применения API-тестирования на практике, отмечая статус выполнения и найденные дефекты.

№	Проверки (Что тестируем?)	Статус прохождения	Ссылка на дефект	Комментарий
Базовые проверки запросов				
1	GET-запрос с валидными обязательными параметрами.			
2	Валидность тела запросов (POST, PUT, PATCH).			
3	Структура URL согласно спецификации.			
4	Параметры при отправке запроса методом GET.			
5	Значения form-data или x-www-form-urlencoded при отправке запроса методом POST.			
Проверка ответов и статус-кодов				
6	Соответствие ответа от сервера требованиям.			
7	Версия протокола в ответе.			
8	Код состояния HTTP (например, 2XX для GET, POST, DELETE, PUT) согласно ТЗ.			
9	Пояснение (status text) в ответе.			
10	Основные заголовки HTTP (HTTP headers) и заголовки ответа согласно требованиям.			
Проверка структуры данных (JSON/XML)				
11	Сформированный объект JSON (либо XML) и его структура согласно модели данных из требований.			
12	Объект в формате JSON с запрошенными полями.			
13	Структура XML ответа от сервера согласно требованиям.			
14	Имена свойств (ключей), типы полей, значения полей согласно требованиям.			
15	Обязательные поля в ответе имеют значения не null, опираясь на требования.			
Проверка безопасности и авторизации				
16	Создание токена авторизации.			
17	Структура полезных данных (payload) токена (bearer token, jwt token).			
18	Различные типы токенов авторизации согласно ТЗ.			
19	Валидность токена авторизации.			
20	Безопасность методов.			https://t.me/protestinginfo/2828
21	Конфиденциальная информация не утекает через заголовки (например, отсутствие X-Powered-By).			
Проверка состояния и идиempотентности				
22	Состояние системы после GET-запросов (отсутствие изменений, идиempотентность).			
23	Идиempотентность для методов PUT и DELETE.			
24	Обновление существующих данных методом PUT.			
25	Обновление существующих данных методом PATCH.			
26	Обновление существующих данных методом POST.			
27	Удаление существующего объекта.			
28	Создание id или uuid при создании ресурсов.			
Проверка производительности				
29	Своевременность ответа на запрос.			
30	Кэшируемость методов.			https://t.me/protestinginfo/2828
Специальные форматы (GraphQL, gRPC)				
31	Отправка валидного запроса в формате GraphQL.			GraphQL examples
32	Отправка валидного запроса в формате gRPC.			Using gRPC examples gRPC test examples

Чек-лист для негативных проверок API

Негативные проверки API фокусируются на том, как система обрабатывает непредвиденные или некорректные входные данные, а также неавторизованные запросы. Их цель — убедиться, что API возвращает соответствующие ошибки и предотвращает нежелательное поведение или уязвимости.

Данный чек-лист поможет создать comprehensive тестовые сценарии, выявить потенциальные недостатки в обработке ошибок и обеспечить надежность API в различных условиях.

№	Проверки (Что тестируем?)	Статус прохождения	Ссылка на дефект	Комментарий
Общие сценарии негативного тестирования				
1	Отправка данных с https://t.me/protestinginfo			Ожидаемый результат при негативном тестировании это должны быть корректный статус, корректное сообщение, и корректное тело ответа от сервера согласно ТЗ
Обязательность полей				
2	Пустые обязательные поля (не допускающие значения null)			Проверка обязательност и полей
3	Пустые значения во вложенных объектах JSON (обязательные поля)			
4	Пустые значения во вложенных массивах JSON (обязательные поля)			
5	Отсутствие обязательного поля			
6	Дубликаты значений обязательных полей в теле запроса			
7	Пустые обязательные HTTP заголовки			
Невалидные и несуществующие значения				
8	Невалидный токен авторизации			
9	Пустой токен авторизации			
10	Несуществующий ID			
11	Невалидный ID			
12	Невалидный UUID в параметрах запроса			
13	Невалидный UUID в теле запроса			
14	Недопустимый UUID			
15	Несуществующий UUID			
16	Невалидные HTTP заголовки			
17	Неподдерживаемые методы HTTP			
18	Невалидное тело в запросе			
19	Невалидный тип параметра			
20	Недопустимые символы в параметрах			
21	Валидные значения параметров (ожидаем ошибку, если они не должны быть валидными в негативном сценарии)			
22	Недопустимые символы в теле запроса			
JSON формат				
23	Недопустимые данные JSON в теле запроса			
24	Неполная JSON модель (отсутствующие поля)			
25	JSON модель с невалидными полями			
26	Полностью пустое тело запроса в формате JSON			
Переполнение данных				
27	UUID длиной порядка 500 символов или больше			
28	ID длиной порядка 100 символов			
29	Переполнение тела запроса (очень большой JSON)			
Специальные сценарии				
30	Запрос с комментариями в теле			
31	Регистрозависимость в теле запроса			
32	Регистрозависимость в заголовках			
33	Регистрозависимость текстовых параметров			
34	Загрузка файла с невалидным форматом через form-data			
35	Отправка фото в сломанном формате base64 через Postman			
Проверка статусов HTTP 4xx				
36	400: Неверный запрос (некорректные действия пользователя)			
37	401: Аутентификация не пройдена			
38	403: Аутентифицированному пользователю не разрешен доступ к указанной конечной точке API			
39	404: Запрашиваемый ресурс не существует			
40	405: Метод не разрешен (проверка заголовка Allow)			
41	410: Запрашиваемый контент удален с сервера			
42	412: Условия, указанные в заголовках, не могут быть выполнены сервером			
43	415: Неподдерживаемый тип медиа (недопустимый тип содержимого или версия)			
44	422: Не удалось выполнить проверку данных (например, в ответ на POST-запрос)			
45	429: Слишком много запросов (ограничение скорости)			
Проверка статусов HTTP 5xx				
46	500: Внутренняя ошибка сервера (имитация и провоцирование необработанных ошибок)			Имитировать и сломать то, что не покрыто в коде
HTTP методы				
47	Невалидные HTTP методы			Нижe ссылки , которые предоставлен ы подписчиком Антоном
Авторизация с использованием методов аутентификации				
48	Запрос без токена авторизации			
49	Запрос с токеном, приводящим к ошибке 403			
50	Запрос с невалидным токеном			
GET запросы				
51	Пустой список (если применимо)			
52	Заполненный список			
53	Пагинация в списке (limit, offset)			
54	Получение списка с ограничением на количество записей			
55	Получение списка, начиная с указанного номера			
56	При отрицательном offset список возвращается, начиная с первой позиции			
57	В случае передачи параметров с некорректным значением возвращается 400 с описанием ошибки в теле ответа			
58	При несуществующем offset возвращается пустой список			
59	Запрос данных по валидному ID: проверка возвращения корректных данных			
60	Запрос данных по несуществующему ID при валидном формате			
61	Запрос данных по невалидному ID			
POST запросы				
62	Все поля заполнены валидными данными			
63	Заполнены только обязательные поля			
64	Не все обязательные поля заполнены			
65	Не заполнено ни одно поле			
66	Валидация данных в полях (невалидные данные)			
67	Пустой JSON			
68	Дата создания объекта			
PUT запросы				
69	Обновление с невалидными данными			
70	Обновление по несуществующему ID			
71	Обновление по невалидному ID			
72	Валидация полей (невалидные данные)			
73	Частичное обновление (в JSON присутствуют не все поля)			
DELETE запросы				
74	Удаление несуществующего объекта			
75	Удаление уже удаленного объекта			
76	Удаление по несуществующему ID			
77	Удаление по невалидному ID			
78	Удаление и повторное добавление такой же сущности (если есть уникальные поля)			
Прочие проверки				
79	Протокол HTTP / HTTPS согласно спецификации: например, Mixed Content: The page at 'https://<->.ru/' was loaded over HTTPS, but requested an insecure stylesheet 'http://fonts.googleapis.com/css2?family=Arimo:wght@400;700&family=Play:wght@400;700&display=swap'. This request has been blocked; the content must be served over HTTPS.			
80	Ошибочные форматы обмена данными (JSON, XML, HTML)			
81	Замена допустимого метода на недопустимый			
82	Большое или недопустимое время отклика			
83	Наличие / отсутствие файла (-ов) куки			

Дополнительные материалы от подписчика Антона

Эти дополнительные материалы, собранные нашим подписчиком Антоном, представляют собой ценную подборку ресурсов для углубленного изучения тестирования API. Используйте их для расширения своих знаний, оттачивания практических навыков и повышения эффективности ваших тестовых процессов. Большая благодарность Антону за эту полезную подборку!

Основные ресурсы

- [Чек-лист API проверок](#) — Подробный чек-лист для всесторонней проверки API, который поможет систематизировать процесс тестирования и ничего не упустить.
- [Postman чек-лист | Postman тестирование API](#) — Видео-урок, демонстрирующий основные функции Postman и лучшие практики для эффективного тестирования API.

Авторизация и аутентификация

- [Authorizing requests \(Postman\)](#) — Официальное руководство Postman по настройке различных типов авторизации для ваших API запросов.
- [How to write authentication test cases with Postman?](#) — Пошаговое руководство по созданию тестовых сценариев для аутентификации в Postman, обеспечивающее безопасность API.
- [Аутентификация в веб-API: пример и ликбез по Postman](#) — Разъяснение различий между аутентификацией и авторизацией, а также практические примеры их реализации в Postman.
- [ТЕСТИРОВАНИЕ АВТОРИЗАЦИИ OAUTH2 В POSTMAN](#) — Глубокое погружение в тестирование авторизации OAuth2 в Postman, незаменимое для работы с современными API.

HTTP методы

- [Тестирование GET-запросов](#) — Руководство по эффективному тестированию GET-запросов, позволяющее проверять корректность получения данных.
- [Тестирование POST-запросов](#) — Подробные инструкции по тестированию POST-запросов, необходимых для проверки создания новых ресурсов.
- [All types of POST Requests with Postman](#) — Обзор различных способов отправки POST-запросов в Postman для максимально полного покрытия тестами.
- [Postman - DELETE Requests](#) — Учебник по выполнению DELETE-запросов в Postman, важный для проверки удаления ресурсов.
- [Postman Tutorial #9 - How to Send DELETE Request in Postman](#) — Видео-урок о том, как правильно отправлять DELETE-запросы в Postman для тестирования функционала удаления.

Специальные темы

- [Как пользоваться Postman: Выполнение запроса](#) — Практическое руководство по выполнению запросов в Postman, охватывающее базовые операции.
- [Постраничная выборка \(limit и offset\)](#) — Объяснение работы пагинации в API (параметры limit и offset) для эффективной обработки больших объемов данных.
- [Выполнение запроса \(Postman\)](#) — Дополнительный материал по выполнению запросов, углубляющий понимание работы с инструментом.
- [Как установить Cookie в Postman. Тестирование Cookie в Postman](#) — Видео-руководство по работе с Cookie в Postman для тестирования сессионных запросов.
- [HTTP статус-коды 500, 502, 503, 504, 505](#) — Объяснение распространенных ошибок сервера (5xx статус-кодов) и их значения для диагностики проблем в API.

Комплексные курсы

- [API для QA: учимся тестированию ПО без доступа к коду](#) — Подробная статья для QA-специалистов, объясняющая, как тестировать API, не имея доступа к исходному коду.

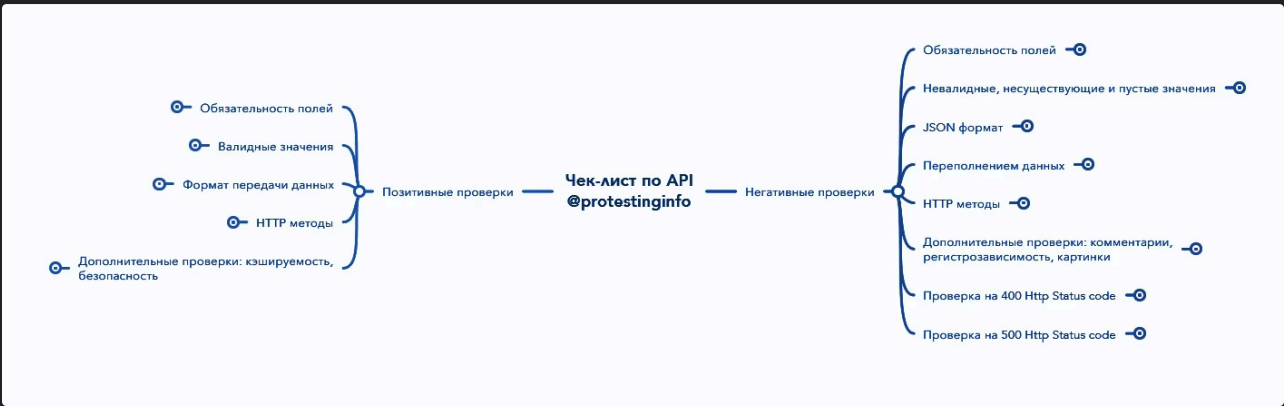
Чек-лист в формате MindMap

MindMap — это мощный визуальный инструмент для структурирования информации, который помогает организовать мысли, планировать задачи и улучшать понимание сложных тем. Этот формат идеально подходит для быстрого обзора ключевых аспектов тестирования и облегчает процесс генерации тестовых сценариев. Он будет особенно полезен QA-специалистам, тестировщикам API и всем, кто хочет систематизировать свои знания в этой области.

<https://www.mindmeister.com/2696738039/api-protestinginfo>

Представляем вашему вниманию интерактивный чек-лист для тестирования API, выполненный в формате MindMap. Он был разработан для помощи в выполнении тестовых заданий и систематизации процесса проверки API. Карта охватывает ключевые аспекты, такие как:

- Аутентификация и авторизация
- Тестирование различных типов запросов (GET, POST, DELETE)
- Работа с пагинацией и куками
- Обработка ошибок
- И многое другое, обеспечивая комплексный подход к тестированию.



Как использовать MindMap чек-лист

Для максимальной эффективности рекомендуем использовать эту интеллект-карту следующим образом:

- **Ознакомление:** Изучите основные ветви и узлы для понимания структуры и охвата тем.
- **Планирование:** Используйте карту как отправную точку для составления планов тестирования, особенно при работе с новыми API.
- **Генерация тест-кейсов:** Каждый узел может стать основой для конкретного тест-кейса или группы тестов.
- **Самопроверка:** Проходите по карте, чтобы убедиться, что не упустили важные аспекты при тестировании.
- **Дополнение:** Карта может служить шаблоном, который вы можете дополнять своими специфическими знаниями и находками.

Этот формат наиболее полезен, когда вам нужен быстрый, но при этом детальный обзор всего объема работ, а также для обучения и онбординга новых членов команды.

Дополнительные ссылки по API

Здесь вы найдете подборку ценных ресурсов для углубленного изучения тестирования API. Используйте эти материалы для расширения своих знаний и оттачивания практических навыков. Подборка включает видеоуроки, вебинары и дополнительные материалы, охватывающие ключевые аспекты работы с API.

Видеоресурсы и вебинары

Начните свой путь в тестировании API с этих подробных видеоматериалов и вебинаров, доступных для просмотра в удобном для вас темпе:

[**API**](#) — Обзор основ API, идеально подходит для начинающих, чтобы понять базовые концепции и принципы работы.

Открыты записи вебинаров — можно смотреть в удобном темпе:

- [**Практический разбор основных способов авторизации в Postman: Bearer Token, OAuth 2.0, API Key и другие методы**](#) — Поможет уверенно работать с различными типами аутентификации при тестировании API и повысить безопасность ваших запросов.
- [**Асинхронность в API: как тестировать Webhook и обрабатывать асинхронные запросы**](#) — Узнайте, как эффективно тестировать асинхронные взаимодействия, чтобы обеспечить надежность и стабильность систем.
- [**Асинхронность в API: как тестировать Webhook \(доп. запись/материалы\)**](#) — Дополнительные материалы и запись, углубляющие понимание асинхронности и методов тестирования Webhook.
- [**Разбор технических вопросов на собеседования QA: тестирование API \(REST API\)**](#) — Подготовьтесь к собеседованиям, освоив ключевые вопросы и ответы по тестированию REST API, чтобы успешно пройти технические этапы.
- [**Вебинар «Разбор вопросов» + тесты**](#) — Закрепите свои знания с помощью интерактивного разбора вопросов и практических тестов, чтобы убедиться в готовности к реальным задачам.

Дополнительные материалы

Изучите дополнительные ресурсы для расширения кругозора и получения актуальной информации:

- [**API**](#) — Ссылка на Telegram-канал с полезными статьями и новостями по теме API тестирования.
- [**Три вебинара по технике тест-дизайна, тестированию API и ситуационным вопросам**](#) — Расширьте свои компетенции с помощью этой подборки, включающей стратегии тест-дизайна и ответы на ситуационные вопросы.

Мои статьи на Habr

Ознакомьтесь с шаблонами тест-кейсов по API и тест-кейсами по проверке идиempотентности в моей статье:

<https://habr.com/ru/articles/718560/>

- Шаблон тест-кейсов API доступен по [ссылке](#).
- Шаблон тест-кейсов API для сайта [Vikunja](#) можно найти по [ссылке](#).
- Тест-кейсы для "Проверки идиempотентности" представлены [здесь](#).

Практическое задание: Автоматизация API-тестирования в Postman

Это практическое задание разработано для углубления ваших навыков в автоматизации тестирования API с использованием Postman. Вы научитесь работать со Swagger-документацией, создавать запросы, парсить ответы и выполнять проверки, что является ключевым для каждого QA-инженера.

Выполнение этого задания позволит вам развить практические умения, необходимые для эффективной проверки стабильности и корректности работы API.

Информация о ресурсе

Вам предлагается работа с коллекцией Swagger по адресу: <https://petstore.swagger.io/>

Для работы используйте следующие предусловия:

- Base URL: <https://petstore.swagger.io/>
- Request URL: `https://petstore.swagger.io/v2/pet`
- Headers: 'content-type': `application/json`
- "status" питомца в теле запроса: `"available"`

Задание

Создайте коллекцию в Postman, содержащую следующие запросы:

Этап 1: Создание питомца

Цель: Отправить данные нового питомца и сохранить его идентификатор для дальнейшего использования.

- Создайте нового питомца (метод `POST /pet`).
- Заполните все поля своими примерами значений.
- На вкладке "Tests" пропишите скрипт, который извлекает ID созданного питомца из тела ответа и сохраняет его в переменную коллекции.

Этап 2: Работа с тегами

Цель: Проверить корректность обработки тегов питомца.

- На вкладке "Tests" пропишите скрипт, который извлекает имя тега (`name tags`) питомца из тела ответа и сохраняет его в переменную коллекции.
- Добавьте проверку (`pm.test`) на соответствие имени тега из тела запроса имени тега из тела ответа сервера.

Этап 3: Получение и обновление

Цель: Извлечь информацию о питомце и изменить его статус.

- Получите информацию о питомце по ID (метод `GET /pet/{petId}`).
- Используйте ранее сохраненную переменную ID из коллекции.
- На вкладке "Scripts" пропишите скрипт, который отправляет следующий запрос "Updates a pet in the store with form data" (метод `POST /pet/{petId}`) для обновления ключа "status" на "sold" (продан).
- Подсказка: используйте `pm.sendRequest`.

Этап 4: Негативные проверки

Цель: Проверить поведение API при некорректных входных данных.

- Создайте дополнительные запросы для следующих сценариев:
- Невалидный ID питомца при создании: ожидается ошибка 400 (bad input).
- Несуществующий ID питомца при получении: ожидается ошибка 404 (Pet not found).
- Неподдерживаемый формат данных при создании: ожидается ошибка 415 (Unsupported Media Type).
- Как спровоцировать ошибку 500 Server Error при создании питомца?
- Ожидаемый ответ:

```
{
  "code": 500,
  "type": "unknown",
  "message": "something bad happened"
}
```

Завершение

После выполнения задания экспортируйте коллекцию (Export/Export Collection) и отправьте на проверку. При желании, вы можете дополнительно разработать тест-кейсы по API.

Контакты

ДО ВСТРЕЧИ В МОИХ БЛОГАХ!

в моем [блоге](#) / [телеграм канале](#) или посетите мой сайт [Protestinginfo](#)

Оповещения по старту курса по тестам и по практике: https://t.me/info_course_protestinginfo